

Expert Review of Hierarchical Clinical Transformer Refactoring with Flash Attention and MoE

Architecture Refactoring Summary

Baseline Hierarchical Model – The original model is a two-level transformer designed for clinical sequences. At the **lower level (intra-day)**, it encodes the multiple medical codes within each day. Each day's code list (up to 80 codes) is embedded and passed through a single-layer Transformer encoder (4 attention heads, small feed-forward) to capture co-occurrence patterns among codes on the same day ¹. The outputs are pooled (via max-pooling) and combined with a simple **sum of code embeddings** (a residual pathway that retains the aggregate signal of all codes) ² ³. Additionally, static demographic features – patient **age** (in months) and **gender** – are embedded and added to each day's representation ³. This forms a per-day vector that represents that day's medical events enriched with demographics. At the **upper level (inter-day)**, a 6-layer Transformer encoder (initially standard self-attention with causal masking to respect chronology) processes the sequence of up to 200 days ⁴. The causal mask ensures the model at day t can only attend to days $\leq t$ (preventing “seeing the future” during training) ⁵. Finally, a linear decoder projects each day's hidden state to the prediction target space (e.g. a probability distribution over possible codes or outcomes for that day) ⁶. The baseline thus **hierarchically models** clinical timelines: first aggregating within day, then modeling across days. This design is well-motivated by the data structure – events in the same day are closely related, and treating each day as a “time step” in a sequence of care allows the model to focus on day-to-day dynamics.

Flash Attention Integration – The refactoring introduces *Flash Attention* into the temporal encoder to improve efficiency. Flash Attention (Dao et al. 2022) is an algorithm that computes the same output as standard attention but uses a fused, tiling approach to avoid explicitly materializing the full $N \times N$ attention matrix ⁷ ⁸. In the context of 200-day sequences with 16 attention heads (head dimension 16), standard attention would form 40k-element score matrices per head, per layer – amounting to ~7.7 million floats stored across 6 layers ⁷. Flash Attention eliminates this memory overhead by processing in blocks, significantly reducing memory access (by about 45× in their case) ⁸. The integration was done in two steps: (1) as a **drop-in replacement** using PyTorch 2.0's native support for scaled dot-product attention with `enable_flash=True` ⁹; and (2) via a **custom Flash-aware Transformer layer** for more control. The custom layer uses **pre-normalization** (LayerNorm applied *before* the attention and feed-forward sublayers, as in GPT-2 and newer models) to improve training stability ¹⁰. It also leverages **PyTorch's F.scaled_dot_product_attention** with the `is_causal` flag for efficiency, so the model no longer manually creates causal masks for flash-enabled layers ¹¹ ¹². In addition, the team introduced **Rotary Position Embeddings (RoPE)** to the temporal attention. RoPE imparts relative positional information by rotating Q and K vectors in a frequency-based manner, enabling better capture of temporal distances and even extrapolation to longer sequences than seen in training ¹³ ¹⁴. This is a smart choice for clinical data, as the importance of events often depends on how far apart in time they occurred (RoPE makes the attention depend on relative difference in positions rather than absolute positions). They also incorporated **SwiGLU** activation in the feed-forward networks (an activation that multiplies one half of the neurons by a sigmoidal gate on the other half) ¹⁵. SwiGLU, used in PaLM and other large models, tends to improve parameter

efficiency and has been shown to provide small accuracy gains over ReLU/GELU. The model is set to train in mixed precision (bfloat16), which pairs well with Flash Attention to maximize throughput. Overall, the flash attention integration modernizes the transformer: more heads (increased from 8 to 16, so each head has smaller dimensionality which meets flash's alignment requirements ¹⁶), pre-norm for stability, and specialized positional encoding. Importantly, these changes **do not alter the model's core outputs or objectives** – they aim to speed up training/inference and potentially improve generalization, but the underlying architecture (hierarchical Transformer) and task remain the same. The **daily code encoder** was left mostly unchanged (still one standard Transformer layer on 80 codes/day), since intra-day sequences are short and the benefit of flash attention there would be minor. They did include a flag to optionally use a flash-based layer for the daily encoder, but by default it remains a regular Transformer ¹⁷ ¹⁸.

Mixture-of-Experts (MoE) Integration – The most significant architectural change is the introduction of MoE layers into the temporal Transformer (the across-days encoder). Instead of each Transformer layer having a single dense feed-forward network (FFN) for all inputs, certain layers now have an *array of expert FFNs* and a learned router that selects a subset of them for each input token (day). Concretely, they replace the **FFN sublayers in layers 2-5** of the 6-layer temporal Transformer with MoE blocks. Layers 0 and 1 (the first two) and layer 6 (the final layer, if counting 1-6) remain standard dense layers ¹⁹ ²⁰. This means the model keeps the initial part of the sequence encoder conventional (possibly to learn general patterns) and the final output layer dense (to produce a coherent output vector), while injecting expert specialization in the middle four layers. Each MoE layer consists of **8 experts** (in the base design) – these are independent feed-forward networks (256 → 512 → 256 dimensions, the same size as the original FFN) that each have their own learned weights. A **gating network** (router) produces a probability distribution over the 8 experts for each token. The router is a simple linear layer that takes the token's hidden state and outputs 8 logits ²¹ ²². For each token (day), the top $k=2$ experts with highest router scores are selected to compute that token's transformation ²³. The token's input is sent through those two experts' FFNs, and the results are weighted by the router's softmax probabilities (renormalized to the top-2) ²⁴ ²⁵, then summed. This output replaces the dense FFN output in that Transformer layer. In pseudo-code: $output = gate1 * FFN_expert1(token) + gate2 * FFN_expert2(token)$. The architecture also supports a concept of **shared experts**: in some experiments, one expert is designated "shared" meaning it is always active for every token (effectively every token always goes through that expert in addition to any routed ones) ²⁶ ²⁷. In such cases, if there is 1 shared expert among the 8, the router only chooses $2-1=1$ additional expert (so each token still uses 2 experts total: the shared plus one routed). The shared expert acts like a baseline expert that captures patterns common to all inputs, allowing the other experts to specialize in more specific patterns.

To train the MoE, they include a **load balancing mechanism**. Without it, the router could learn pathological solutions (like always picking the same expert for everything, or some experts never getting used). They implemented the **Switch Transformer auxiliary loss** which penalizes mismatch between the router's average assignment probability and actual usage of each expert ²⁸ ²⁹. This loss $L_{\text{aux}} = N \sum_i p_i \times \text{usage}_i$ (for N experts) is minimized when each expert's probability p_i and usage fraction are both $1/N$, i.e. all experts equally utilized ²⁸. They set the auxiliary loss weight to 0.01 (a typical small coefficient) so that it nudges the router without overpowering the main training objective ³⁰. Additionally, they integrated an **auxiliary-free balancing** method inspired by "DeepSeek-V3" (2024). In that approach, instead of adding a loss, the router logits have a learned bias that is adjusted after each batch to push expert utilization toward uniformity ³¹ ³². If an expert was overused in a batch, the bias for that expert is decreased slightly (making it less likely to be chosen next time), and vice versa for underused experts ³². This method aims for the same effect (balanced usage) but without explicitly backpropagating

a loss for it, avoiding possible gradient interference with the main task. They plan to experiment with both strategies: the default (Switch) and the bias-correction approach (DeepSeek) ³³.

Importantly, the MoE integration is done **with careful control of other factors**. The overall model hidden size (256) and attention mechanism remain the same, so any improvement comes from the increased model capacity and specialization in the FFN layers. The team outlined multiple configurations to explore the impact of MoE thoroughly. The base setup (Exp 2 in their plan) is 8 experts, top-2 routing, no shared expert ³⁰. Another variant (Exp 3) uses **1 shared + 7 experts**, with still top-2 selected (meaning effectively 1 shared always + 1 out of 7 chosen) ³⁴. This tests if having a global expert for common patterns improves performance or stability. A **fine-grained MoE** variant (Exp 4) increases the total number of experts to 16 (1 shared + 15) but *reduces each expert's size* (their FFN hidden dim from 512 down to ~238) ³⁵. This keeps the total parameter count per layer roughly constant (so the MoE layer isn't using more parameters than a dense one, just distributing them differently) – a very thoughtful control to isolate the effect of having more *experts* versus just more capacity. It also allows more specialization, since 15 smaller experts could each focus on a narrower aspect of the data. Finally, an **auxiliary-free MoE** variant (Exp 5) uses the shared expert design but replaces the Switch aux loss with the DeepSeek bias mechanism (and sets the aux loss weight to 0) ³⁶ ³⁷. This will show if the new balancing method can achieve similar or better performance without the extra loss term. All these configurations will use the same overall architecture otherwise, enabling apples-to-apples comparisons.

To summarize, the refactored architecture retains the hierarchical Transformer backbone but incorporates two advanced techniques: **Flash Attention** for faster and memory-efficient sequence processing (with improved positional encoding and activation functions), and **Mixture-of-Experts** for increasing model capacity via sparsely-activated feed-forward layers. The expectation is that Flash Attention will allow scaling to longer sequences or larger batches with less computational cost, and MoE will improve predictive performance (especially on heterogeneous or rare-event data) by allocating model capacity dynamically to different portions of the sequence data.

Critical Assessment of Design Decisions

Hierarchical Modeling of EHR – The decision to use a hierarchical encoder (per-day and across-days) is well-justified and aligns with best practices in the domain. EHR data is naturally segmented into visits or days, and treating each day's codes as a set to be summarized prevents the sequence length from exploding (otherwise a patient with 200 days and 80 codes/day would be a single sequence of 16,000 tokens, which is both computationally heavy and conceptually different – it would treat every code like a step in a long sentence). Prior research in clinical NLP has similarly found that a hierarchical approach can improve modeling of long medical records ³⁸. By extracting local (within-visit) features and then feeding those into a higher-level sequence model, the architecture leverages the strong local correlations of medical events (e.g., codes on the same day often relate to the same clinical episode) ³⁸. The baseline's implementation of this – one transformer layer per day and pooling – is a reasonable trade-off between complexity and expressiveness. One potential critique is whether a **single-layer** daily encoder is sufficient to capture complex interactions among codes. They mitigated this by also using the simple sum of code embeddings as a residual pathway, which means the day representation contains both an "interaction-aware" component (from the Transformer + max-pooling) and a straightforward "bag of codes" component ² ³. This is a clever design: the sum of embeddings gives a quick, holistic summary of which codes are present (useful for rare codes that maybe the transformer layer could overlook), while the transformer can focus on any higher-order combinations (e.g., particular code co-occurrences) and temporal ordering of

codes if relevant. Max-pooling after the code-level transformer ensures that the most salient code embedding (perhaps the one that got the highest contextual activation) is preserved, which is a form of attention by position (though non-learnable). An alternative could have been an attention-pooling or using the transformer's output of a special "[CLS]" token per day, but max-pooling is simple and effective to get an aggregate.

Given that, the combination `day_vector = sum(codes) + pooled_transformer(codes) + demographics` ensures each day's vector has multiple sources of information fused.

Including **age and gender embeddings per day** is important for clinical modeling – these features can heavily influence risk (e.g., certain predictions vary greatly by age/gender). By adding them to each day's representation, the model effectively reminds itself of the patient's demographics at every time step. One might argue age changes over time; their approach of using age in *months* and capping at 120 years (1440 months) is granular enough and they do update the age each day (since presumably the "age_in_months" value increments as the sequence goes forward) ³⁹ ⁴⁰. So the model knows how old the patient was on each day, which is excellent. The gender typically doesn't change, but embedding it per day is fine (it's a constant added each time; conceptually it might have been cleaner to add gender only once or as part of initial embedding, but it doesn't harm to add it repeatedly). Overall, the hierarchical design decisions show a strong understanding of the problem structure and are consistent with approaches used in recent literature (e.g., hierarchical transformers like Hi-BEHT in clinical NLP used a similar idea of visit-level and patient-level Transformers).

One minor consideration: the model treats all days up to 200 with equal maximum sequence length, padding if needed ⁴¹. This means if a patient has fewer than 200 days of data, the rest are padded out. The causal mask and `dt_cnt` ensure padded days don't contribute to loss ⁴² ⁴³. This zero-padding approach is standard, though somewhat inefficient if there is high variance in sequence lengths. An improvement could be to use dynamic batching by sequence length or a packing technique, but that's an engineering detail. It doesn't affect the design correctness – it's a pragmatic way to batch data.

Flash Attention and Related Optimizations – Adopting flash attention in the temporal encoder is an excellent choice for this project. It directly addresses a known bottleneck: the quadratic memory and time complexity of standard attention. The provided analysis in the design documents quantifies this well – for sequence length 200 and head dim 16, the attention score matrix is 40k elements, which across heads and layers balloons into millions of elements ⁷. Flash Attention's algorithm avoids storing these matrices at once, leading to drastically reduced memory usage without approximation error ⁴⁴. This will allow the model to potentially scale beyond 200 days if needed, or use larger batch sizes, or simply run faster because it better utilizes GPU memory hierarchy (stays in high-speed SRAM as much as possible) ⁴⁵ ⁴⁶.

The integration via PyTorch 2.0's `scaled_dot_product_attention` is straightforward and robust – it automatically uses the fastest kernel (flash, math, or memory-efficient) depending on hardware and input sizes ⁴⁷. They explicitly enabled flash and disabled the others to force usage, which ensures maximum speed ⁹. They also checked conditions like GPU capability (≥ 7.5) and `head_dim % 8 == 0` ¹⁶, which the model meets ($256/16=16$, divisible by 8). If someone were to run this on an older GPU (pre-Turing), it would gracefully fall back to normal attention, albeit slower – this is acceptable.

Crucially, they didn't stop at just making it faster; they also improved the **training dynamics** with *pre-normalization*. Pre-norm means each sublayer (attention or FFN) is preceded by LayerNorm on its input, and the residual connection is applied to the *output without an extra norm* (except maybe a final norm after the

last layer). This contrasts with the original PyTorch Transformer, which uses post-norm (LayerNorm after adding the sublayer output to the residual). Pre-norm is known to ease optimization for deep Transformers because the gradients can flow through the normalized paths more easily and it mitigates issues of residual block scaling ¹⁰. Many modern Transformer architectures (like GPT-2, etc.) use pre-norm for precisely those reasons. Since their model is only 6 layers deep, it might train okay either way, but pre-norm is still a safer default and can converge faster or to a better optimum. They even mention possibly better initialization (scaling some weights) to accompany pre-norm ⁴⁸ ⁴⁹, which is another sign of thoroughness – initializing the output projection weights to zero, for instance, helps the residual connection at the start of training (the layer starts as essentially identity before training, which can prevent sudden large changes early on) ⁵⁰ ⁴⁹.

Integrating **RoPE** is a strong design decision for a couple of reasons: (1) The baseline seemingly did not use any explicit positional encoding for the day sequence (the code didn't show adding a positional index embedding for days, and in absence of that, the PyTorch Transformer would treat the sequence as unordered aside from the masking). If that's true, then previously the model might have had to infer some notion of time purely from the mask (which is possible but not ideal – the mask prevents future info but doesn't distinguish how far in the past something is). RoPE will give the self-attention a sense of relative distance: e.g., an event 1 day ago vs 30 days ago will produce a different phase in the QK dot product ⁵¹ ⁵². In clinical terms, recency is important (a hospitalization last week is more relevant than one years ago for predicting tomorrow's events). RoPE encodes that smoothly into the model. (2) RoPE enables extrapolation: if they train on sequences up to 200 days and later want to apply the model to 250 days, RoPE can handle it by extending the rotations (unlike learned positional embeddings which would be limited to the training max). This future-proofs the model to some extent. All of these benefits come at virtually no cost (it's a small computation and no learned parameters), so it's an obvious win. Traditional learned positional embeddings could also work, but RoPE has become popular in large language models for its nice properties, so it's sensible here.

The use of **SwiGLU** activation in place of ReLU/GELU in the FFNs is another modern tweak. SwiGLU (Swish-Gated Linear Unit) essentially splits the intermediate layer and uses one part to gate the other (multiplying a *swish* activation with a linear transform) ⁵³. Empirical studies (Shazeer 2020) found it can slightly improve model quality for the same parameter count, at the expense of needing to double the hidden dimension (because half the neurons are used for gating). In their Flash implementation, if they enabled `use_swiglu=True`, they likely adjusted the FFN hidden size accordingly and implemented the gating. This will increase the FFN parameters by ~2x, but those are still relatively small compared to embeddings. It's a reasonable trade-off especially if they observed better validation loss with it. However, one should note that enabling SwiGLU doubles the computations in the FFN. If they did use it, the base FFN hidden of 512 might effectively become $2562=512$ for two linear matrices, gating half of them (resulting in 256 output dims) – or if they kept 512 as output, the intermediate would be 768 (5121.5) as some implementations do. Without their exact code, we assume they adjusted properly. In any case, it's a fine enhancement that aligns with state-of-the-art Transformer designs (e.g., PaLM used SwiGLU and found gains).

Potential issues with Flash integration: Very few, since FlashAttention is designed to be mathematically identical to standard attention (assuming no numerical precision issues). They even planned tests to verify numerical equivalence to ensure the refactor didn't introduce bugs ⁵⁴. One thing to monitor is that the flash kernel's dropout behaves the same as the standard one; they pass the dropout probability (0.1 during training) into `scaled_dot_product_attention` which will internally handle it in the fused manner ⁵⁵. That should be fine. Another consideration: by using flash, they rely on PyTorch's implementation – if any

future PyTorch version changes or if they export the model to ONNX, they'd need to ensure compatibility. But these are minor and can be handled by falling back to standard attention for deployment if needed. The integration seems well-contained and beneficial.

Mixture-of-Experts Design Choices – The MoE design is ambitious and brings in a lot of moving parts, but it is grounded in established research (Switch Transformer ⁵⁶, GShard, etc.) and is adapted thoughtfully to this problem. Here are assessments of specific decisions:

- *Layers 2–5 as MoE*: This means the model has dense feed-forward layers in early and late positions, with MoEs in the middle. This is likely to ensure *stability* and *expressivity*. The first layer or two handle embeddings and initial interactions; keeping them dense avoids the complication of routing right off the bat (when the model hasn't formed a rich representation yet). The last layer being dense ensures that the final output for each token is a single vector integrated from a common feed-forward network, which might make the output distribution more coherent. If the last layer were MoE, one could imagine a scenario where, say, Expert 3 in the last layer produces very different scaled outputs than Expert 7, which could make the final logits distribution vary depending on which expert was chosen. By ending with a dense layer, they ensure a unified processing of the combined expert outputs from the previous layer. Google's Switch Transformer paper often made *all* layers MoE (except maybe embedding layers), but they were working at very large scale where they could handle the complexity. Given this is a smaller model and a novel domain, this partial MoE approach is prudent – it reduces the number of router/expert weights that need to be trained and possibly avoids too many routing decisions compounding.
- *Number of Experts (8)*: The choice of 8 experts is a moderate one. It increases the parameter count of those FFN layers by 8× (minus any shared ones) compared to a single FFN. For context, a single FFN layer (256→512→256) has about $256 \times 512 + 512 \times 256 \approx 262k$ weights. Eight of those is ~2.1 million per layer. Four MoE layers thus add ~8.4 million weights, which is substantial but not outrageous for modern GPUs. In fact, the embedding layer (84010 vocab * 256 dim) is ~21.5 million parameters, larger than the entire MoE addition. So parameter-wise, 8 experts is quite reasonable. They also plan to test 16 smaller experts – that configuration keeps parameters roughly constant (approx 2.1M per layer) ³⁵, which is a clever way to see if *diversity of experts* helps more than just *size of experts*. If the 16-expert model performs better than the 8-expert one, it suggests many specialized brains beat a few bigger ones (holding capacity similar). If not, it suggests diminishing returns or that each expert became too weak.
- *Top-2 routing*: They decided on each token consulting 2 experts. This is in line with the MoE literature where using 2 experts can yield better results than 1 (Mixture-of-Experts in GShard used top-2). The advantage is that each token's representation benefits from two different “opinions” – if one expert alone might not capture everything, the second can complement it. It also means more experts get activated per token (increasing utilization and gradient flow across experts). The downside is doubling the computation per token for those layers, but since they only do 4 MoE layers, that's like adding the cost of 4 extra FFN computations per token per forward. It's still far less than, say, doubling the number of Transformer layers. So it's a good trade-off. Top-1 (as in Switch Transformer) would be more sparse and efficient, but can suffer if a token really belongs to multiple clusters or if the chosen expert is imperfect. Top-2 tends to smooth out these issues. They will need to ensure their implementation of combining two experts is efficient; the provided code snippet does a loop over experts and tokens ⁵⁷ ²⁵ which is okay for moderate batch sizes but might become slow if

not optimized (PyTorch doesn't have built-in sparse dispatch for MoE, though DeepSpeed MoE and others exist – not sure if they used those). Given batch size and sequence length, number of tokens per batch might be on the order of $16 * 200 = 3200$ tokens (if batch 16). That's fine for Python loops. If they scaled up batch or sequence a lot, they might want to vectorize it (e.g., use scatter operations), but performance was likely acceptable so far.

- *Shared Expert*: The idea of a shared expert always active is interesting. This effectively provides a *dedicated capacity* that every token goes through, separate from the router choices. This could act as a sort of baseline or “generalist” expert that captures patterns that are global (not easily partitioned by the router). The expectation is that the shared expert might take care of very common transformations (for example, maybe general language or medical knowledge that applies everywhere), while the routed experts handle more niche situations. They test this in Exp3 and Exp5. I suspect this comes from the DeepSeek methodology they cited – perhaps they found that having one expert that always sees all data improves consistency. I agree that this can prevent the scenario where the MoE model forgets some globally important features because each expert is too specialized. By always including the shared expert's output (they average if more than one shared, but usually it's one) ²⁷, every token has a component of its representation that is learned from *all data*. This might also help with training stability early on: at initialization, the router might be random and some tokens could get weird experts, but the shared expert provides a constant path so the model can still make some progress on all tokens. It's a bit analogous to having a residual path that doesn't depend on routing (except here it's a learnable residual via the shared expert). A possible downside is the shared expert could dominate if the router doesn't find useful partitions – but with proper load balancing, the model will still utilize the others. Overall, it's a sensible experiment to include; it's not standard in all MoE implementations, so it's a novel twist that could pay off.
- *Routing and Load Balancing*: They correctly identified load imbalance as a primary concern. The Switch Transformer aux loss they implemented will encourage the router to distribute tokens roughly evenly among the 8 experts ²⁸ ⁵⁸. However, it's known that this aux loss can sometimes conflict with the main loss – it might force a slightly suboptimal routing just to satisfy the balance, possibly hurting perplexity a bit (Fedus et al. noted this). That's why the DeepSeek bias idea is appealing: it tries to achieve balance *outside* of the main optimization loop by adjusting biases. This is a more decoupled approach. It's relatively new and unproven in literature (the reference is a 2024 technical report), but if it works, it could be quite impactful. They'll measure things like **expert load** (fraction of tokens per expert) and we expect with either method, they aim for ~12.5% per expert (for 8 experts) with low variance ⁵⁹. I appreciate that they planned metrics for this: e.g., standard deviation of expert load (to quantify imbalance) and router entropy (to see if the router is too confident or keeping options open) ⁵⁹. Those choices demonstrate research rigor – they're not treating MoE as a black box, but actively monitoring if it behaves as intended.
- *Expert Capacity and Activation*: With top-2 among 8, at most $28=16$ expert computations per token across all experts (if every expert got two tokens, in theory). In practice, some experts might handle more tokens than others. One thing not explicitly mentioned but likely done: *capacity factor** – often MoE implementations limit how many tokens an expert can take per batch to avoid one expert getting overloaded and becoming a bottleneck. The provided code appears to process each expert's tokens sequentially without a cap ⁶⁰ (other than skipping if none). For their scale, this is fine; capacity factor matters more in large batches where one might impose that an expert can only get, say, $1.5\times$

($\text{top_k} * \text{batch_size} / \text{num_experts}$) tokens and the rest get routed to second-best or dropped. They might not need that here if batch size is small and the aux loss keeps things balanced.

- *Parameter Count vs Performance:* By adding MoE, the model's parameter count increases significantly. Roughly, baseline had ~22M (most in embeddings). MoE adds ~8M as mentioned. So maybe ~30M total now. That's still small in modern terms, but noteworthy. If this yields, say, a 5-8% performance gain as they expect ⁶¹, it's a good trade. It's also computationally efficient because those extra parameters aren't all used at once (only 2 experts active per token). They effectively increased model capacity without proportional increase in computation – the essence of MoE's appeal. This should allow the model to capture more patterns, especially in a domain like healthcare where the feature space (possible codes and combinations) is huge and data can be sparse for many patterns. An expert can specialize on a subset of the distribution (for example, one expert might handle sequences where the patient has diabetes, another for sequences with pregnancy-related codes, etc.). The expectation (which they explicitly plan to test via the “rare code accuracy” metric) is that MoE will improve prediction of infrequent events ⁶². The rationale is that one or a few experts can focus on those rarer patterns without being diluted by having to also model all the common patterns. This **scientific hypothesis** is very plausible. If results show an outsized gain on the tail of the distribution (rare codes) with MoE, that would be a valuable finding.
- *Implementation complexity:* The MoE implementation in `moe_1.py` is quite complex, but from what we see, it's methodical. They rolled their own rather than using a library like DeepSpeed MoE or FairSeq's MoE, likely for learning and flexibility. This carries some risk (it's easier to make a mistake in combining gradients or indices), but the code is clear. They flatten the [seq, batch, d_model] to [tokens, d_model], compute router softmax, pick top-k, and then for each expert gather its tokens ⁶³ ⁶⁴. The use of Python loops could be optimized, but for moderate sizes it's acceptable. One improvement in code could be vectorizing the expert gating accumulation (the inner loop where they assign gate weights for each token) ⁶⁵ – that could be done with advanced indexing in PyTorch (since `top_k_indices` already tells which expert each token got for each of top-k positions). However, clarity might have been chosen over micro-optimization. The shared expert forward pass applies to all tokens (no routing needed) and they simply average its output across however many shared experts ²⁷, which is fine.

In terms of *design robustness*: combining Flash Attention and MoE is ambitious because both are relatively new features that can have their own issues. FlashAttention should not change model outputs except for floating-point rounding differences, so it shouldn't affect MoE routing in any systematic way (just faster). One thing to note: FlashAttention as implemented in PyTorch does not support some forms of attention bias (except causal masks, which are fine) – they explicitly avoided any trainable attention biases so that's not an issue ¹⁶. MoE mainly affects the feed-forward part, which is independent of attention mechanism. So architecturally, these two enhancements don't interfere with each other, which is good. The overall design is quite cutting-edge for an application in healthcare: not many works in 2023/2024 would have combined hierarchical Transformers with MoEs and flash attention, so this is certainly pushing the envelope in terms of technique.

Design Decision Summary: From an AI researcher/engineer perspective, the refactoring decisions align with **state-of-the-art practices** in sequence modeling and scaling: - They preserved the hierarchical structure (sensible given the data) and improved its components (added positional encoding where it was missing, used better activation and normalization). - They upgraded the *efficiency* by using FlashAttention –

a no-brainer for long sequences on GPU – which will help them handle sequences and training more effectively. - They scaled the *capacity* by using MoE – a principled way to get a bigger model without a proportional increase in computation, which is ideal for potentially large clinical datasets or very large code vocabularies. - They paid attention to *regularization and balance* (aux loss, etc.) to ensure the MoE actually works in practice and doesn't collapse.

Every design choice appears well-informed by either literature or known best practices. There are always alternatives (we'll discuss some possible improvements in the next sections), but nothing stands out as a poor choice. The main caveat is that this is a complex model now; training it will require careful tuning to realize the benefits of these choices. But as far as architecture goes, they have assembled a potent combination of techniques that, if executed well, should yield a high-performing and efficient model.

Scientific Validity and Experimental Rigor

Soundness of the Approach – The incorporation of FlashAttention and MoE is grounded in solid prior research and addresses clear needs: - *FlashAttention* is a recently validated method (with proofs of equivalence to standard attention) that targets efficiency, which is crucial given the model's sequence length. It doesn't change the theoretical capabilities of the model, just how fast and memory-efficient it is. Using it does not raise concerns of correctness – indeed the authors provided justification that it produces *identical results* to standard attention by performing an exact softmax computation in blocks ⁴⁴. This means the scientific claims of any results are not compromised by approximation. The use of flash attention should therefore be seen as an implementation improvement that allows the experiments to scale, rather than a speculative research idea – it's a solid engineering choice that is increasingly common in state-of-the-art transformers. - *Mixture-of-Experts* in transformers has a growing body of evidence (e.g., Fedus et al. 2021, Lepikhin et al. 2020) showing it can massively increase model capacity and often improves performance, especially on tasks with diverse or multitask data. In this case, clinical data can be thought of as having many “sub-tasks” (predicting different kinds of events, dealing with different subpopulations of patients, etc.), which is exactly where MoE tends to shine. The user's plan to apply MoE is scientifically valid: it's not guaranteed to work, but it's a hypothesis grounded in literature. They explicitly outline research questions for each experiment (e.g., does MoE improve over dense? does a shared expert help? etc.), which shows they are treating this as an empirical investigation ⁶⁶. This hypothesis-driven approach is a strong aspect of scientific rigor.

Comprehensiveness of Ablations – The planned experiments are impressively comprehensive. The five experiments cover: 1. **Baseline Dense** – as a control. 2. **Standard MoE (8 experts, top-2)** – test if MoE (in the configured layers) improves performance over baseline. 3. **Shared vs Routed MoE** – test if adding a shared expert (common pathway) further improves performance over standard MoE. 4. **Many-Small Experts vs Fewer-Large** – test if increasing expert count (with smaller individual capacity) helps, which examines the effect of expert *diversity* under a fixed or similar computational budget. 5. **Auxiliary Loss vs Bias Balancing** – test if the new load balancing method can match or exceed the traditional method.

Each experiment changes one main factor at a time ⁶⁷ ²⁰. This is a textbook example of controlled experimentation in ML. By isolating variables, they will be able to attribute performance differences to specific changes: - If Experiment 2 outperforms 1, that validates the MoE approach in general (with the given config). - If Experiment 3 outperforms 2, that specifically indicates the shared expert contributed something useful ⁶⁸ ⁶⁹ (maybe capturing common info). - If Experiment 4 outperforms 3, then using more (but smaller) experts was beneficial, suggesting the model was under-fitting or that specialization

helps beyond a certain point. - If Experiment 5 performs comparably to 3 (or better in stability), it means the bias correction is a viable replacement for aux loss.

They also hold constant many things across experiments: same data, same training schedule, same random seed, same model hidden size, etc. ⁷⁰. This greatly increases confidence that differences observed are due to the intended changes and not confounders. The use of a fixed random seed is useful for direct comparison, though for final results they might still run multiple seeds (to get a sense of variance). But for the ablation chain, keeping the seed the same means differences in training are entirely due to the model change, which is a clever trick to reduce noise in comparisons. (One has to be careful: using the same seed for MoE vs dense means even the data order and initialization are identical, so it's fine. The stochastic paths in MoE might still introduce some randomness if any race conditions, but presumably not.)

The only slight addition one could suggest is comparing to a **scaled-up dense model** to separate the effect of *sparsity* from just *more parameters*. For example, if MoE experiment (with top-2) uses ~2x the compute per token in those layers, one might compare to a dense model with, say, double the FFN size or more layers (to equalize compute or parameter count). However, Exp4 somewhat addresses the parameter count question by keeping MoE param count similar to baseline and seeing if it still gains ⁷¹. If Exp4 yields improvement at equal parameter count, that's strong evidence that MoE's *sparse allocation of capacity* is more effective than a uniform allocation. If it doesn't, then perhaps MoE's improvement was largely due to increased parameters rather than specialization. In any case, their design is clever enough to get at this question.

Training Feasibility - The plan seems feasible on available hardware. They mention using 4 GPUs (DataParallel) ⁷², which indicates they have access to multi-GPU setups. The batch size is 16 (likely per iteration; possibly per GPU or total - it's not entirely clear, but likely total batch 16 distributed across 4 GPUs = 4 per GPU). A batch of 16 sequences of length 200 with this model is not extremely large. With flash attention, memory per sequence is low. The MoE part might consume extra memory because of the experts - effectively each MoE layer is 8 separate FFN weight matrices. But 8 FFNs of size 256x512 is nothing a modern GPU (with say 16GB memory) can't handle. The routing computation adds some overhead (storing router_probs for each token - 3200 tokens * 8 = 25,600 probabilities, trivial; some indexing tensors, etc.). So memory-wise, they should be fine even with MoE.

Computation-wise, flash attention will drastically reduce attention time. The MoE will increase FFN computation: normally each token does one FFN (256→512→256 ~ 262k multiplications). With top-2, it does ~524k multiplications (two FFNs) per token in those layers. Across 4 MoE layers, that's an extra ~1 million multiplications per token. For 3200 tokens, that's 3.2 billion multiplications. This is definitely a load, but since FFNs are highly parallel matrix ops, GPUs handle it well. Moreover, some tokens might effectively use slightly less if one of their chosen experts is smaller (like in Exp4, where the 15 routed experts are smaller). Also, if using half precision, these operations are fast. In summary, training will be slower than baseline, but likely still within practical range. If baseline took X hours, MoE might take, say, 1.5-2X hours for the same epochs, which is acceptable given the anticipated performance gains.

One feasibility aspect is the *router training*. Sometimes MoE routers can cause instabilities (spikes in loss if an expert suddenly gets overloaded). The presence of gradient clipping (set to 1.0) ⁷³ is reassuring - that will cut any huge gradient bursts that might come from gating decisions. Additionally, the authors' plan to use mixed precision and possibly BF16 is good, but one should watch out that BF16 can sometimes cause overflow in softmax if not careful. PyTorch's softmax on BF16 is usually fine since it may internally use FP32 for the exponent. Their code likely defaults to FP32 for router softmax (since they didn't indicate custom

implementation). The DeepSeek bias update is outside the gradient system (it's like a small iterative adjustment each step) ⁷⁴. That's fine but they should ensure the bias doesn't drift too far (the magnitude of bias should ideally stabilize when loads are balanced). The bias update rule looks like a simple moving average with momentum, which should converge.

Overall, I anticipate that **training this model is feasible but will require careful monitoring**. The plan to log internal metrics (expert usage, etc.) every 10% of training ⁷⁵ ⁷⁶ is very helpful. They will catch issues like one expert never getting used, or training loss diverging. They also mention validating numerical stability – ensuring flash and non-flash produce same outputs on test examples ⁵⁴ – which is wise to build trust in the new implementation.

Deployment Considerations – From a deployment perspective, the flash attention + MoE model is more complex than the baseline, but still deployable: - FlashAttention on inference: PyTorch 2.x will use flash attention on GPU by default if conditions are met, so inference will also be fast. If deploying on CPU or older GPU, it will fall back to normal attention, which could be slower but still functional. If latency is a concern, one might restrict use to GPU inference. - The MoE is perhaps the bigger consideration for production. Typical deep learning frameworks don't have native MoE support, so exporting to something like ONNX might be challenging due to the dynamic control flow (the loop over experts and index selection). However, one can deploy the model *in PyTorch* (or TorchScript it). TorchScript can handle some amount of control flow. The for-loop over experts could be unrolled if the number of experts is fixed (8 is fixed) – one could write the MoE forward in a way that uses scatter/gather so it's more data-parallel and TorchScript friendly. If not, running the Python logic in production is possible but not ideal for latency (though 8 experts loop isn't too bad). - That said, for an initial deployment or internal use, a Python-based model on GPU might be fine. If they anticipate high throughput, they might consider using a MoE library or custom CUDA kernels to do the routing faster. But those are optimizations not necessary unless this goes into a real-time clinical decision support system.

Another deployment aspect: memory footprint. With ~30M parameters, the model is maybe 120 MB in fp32, or ~60 MB in fp16. That's very lightweight for a GPU server, so no issues there. The bigger memory use is at inference time, if one were to score a very long sequence – but 200 days isn't huge. And flash attention means memory usage scales linearly, so even if 200 -> 300 or 400, it's fine.

One nice thing is that the MoE allows **scaling up** the model further if needed for deployment without a complete redesign. If they found they need more capacity, they could increase experts to 16 or 32 and deploy on multiple GPUs (each GPU hosting a subset of experts, as done in some MoE serving scenarios). That might not be needed here, but the architecture is flexible in that sense.

In terms of **scientific reproducibility and validity**: The team appears to be following good practices. They've documented the architecture in detail, plan to measure a variety of metrics, and even mention external evaluation (using the learned representations in downstream tasks) ⁷⁷. This is excellent because it checks if the model's improvements translate to real clinical utility (e.g., using the transformer output as features for risk prediction tasks). They explicitly call out focusing on *clinically relevant metrics* like top-5 and top-10 accuracy rather than just perplexity ⁷⁸ ⁷⁹, which shows an understanding of the domain's needs (doctors care if the correct code is somewhere in the top suggestions, not necessarily ranked first). From a publication standpoint, this focus on appropriate metrics and demonstrating improvement in them is crucial for convincing readers that this isn't just a theoretical exercise, but actually yields practical benefits. The inclusion of metrics like **MRR (Mean Reciprocal Rank)** and **Rare Code Accuracy** also indicates

thoroughness – they’re checking not only overall accuracy but how well the model ranks the true code and how it does on rare subsets of codes ⁸⁰ ⁸¹ . This aligns with their hypothesis that MoE might help with tail performance ⁶² .

One aspect to validate scientifically is whether the improvements are **statistically significant** and not just due to chance or increased parameters. Given their careful controls, if they see, say, a +5% to +8% gain with MoE (as expected) ⁶¹ , that’s likely well beyond any random fluctuation (especially if they use a large validation set). They might still run a significance test or at least multiple seeds to confirm consistency. For top conferences, reporting an average over 3 runs with standard deviation is common. It’s not mentioned in the plan, but they might do that eventually. If MoE’s gain is smaller (say 1-2%), that might be within the noise, which would need more careful validation. But their expectation of 5-8% seems based on literature or intuition and sounds reasonably attainable if the task is complex enough.

Publication Quality – Judging by the detailed plans and modular code, the work is being conducted in a very systematic, *research-oriented* manner. They are clearly aiming for a publication (the mention of NeurIPS, ICLR, etc. confirms this). To meet that bar: - The **novelty** should be highlighted. On first glance, one might say “FlashAttention is known, MoE is known”. However, applying these in the clinical domain with a hierarchical model could be novel. If they provide insights like “MoE drastically improves rare event prediction in EHR data” or “the combination of flash attention and hierarchical modeling enables us to train on whole patient histories that were previously intractable”, those are valuable contributions. Also, if the DeepSeek bias method isn’t widely known, demonstrating it in action could be a contribution (especially if it proves superior or even just equal but simpler than aux loss). - The **writing and structure** of their documents (like the experiment plan) is already at a high level – it reads like a methods section of a paper with clear tables and reasoning. This is a good sign for publication readiness. They have references to literature in those docs (e.g., citing Switch Transformer, citing BEHRT for metrics) ⁷⁸ ⁸² , showing they’re positioning the work in context. - They are covering all bases: efficiency (with flash), effectiveness (with MoE), and even **interpretability** to some extent (via analyzing expert usage, etc.). For a top ML conference, one usually needs either a strong theoretical innovation or a strong empirical showing on important problems. This work leans towards empirical/modeling innovation in an important domain (healthcare). If they can show that their model outperforms previous state-of-the-art on a benchmark task (like risk prediction or next visit prediction), that would make a compelling case. It’s important they compare to baseline methods (like maybe an RNN or a vanilla Transformer without hierarchy, or existing published models on the same data if available). They did mention BEHRT and others in metrics context; hopefully they’ll actually compare against them or at least use them as references for what metrics values are considered good ⁷⁸ . - The plan to evaluate on **downstream tasks** (external evaluation) is excellent. For instance, they might train the model (self-supervised or supervised on next-code prediction) and then use the patient embedding (maybe the final day’s hidden state or an aggregation of all days) for a task like predicting onset of a disease in the next year. If their learned representation leads to higher AUC or accuracy on such tasks compared to baseline, that’s a big win and very publishable (since it shows transfer learning potential and clinical relevance). - They have to be cautious in how they present the complexity: reviewers might ask “The model is so complex – flash attention, MoE, hierarchical – how do we know which part contributed what?” They are ready for that with their ablations. They might also ask “Is this overkill for the problem? Would a simpler model achieve similar results?” To preempt that, they should compare with simpler models (like baseline transformer, maybe even logistic regression or Random Forest on aggregated features, to show the transformer indeed is needed). Given BEHRT (a prior model) had certain performance, they should show they exceed that. Their focus on rare codes suggests maybe baseline had trouble with those, which MoE fixes – that narrative would be very compelling (it addresses a known challenge in clinical

prediction: common events are easy, rare events are hard but often critical, e.g., predicting a rare disease). - Another angle: **Generalization and robustness** – if they can demonstrate that the MoE model is more robust to something like distribution shift (for example, if tested on a slightly different patient cohort, does it maintain performance better than baseline?), that would be a bonus. MoEs sometimes can adapt better because experts can specialize in new patterns if fine-tuned. They didn't explicitly mention this, but it could be explored.

In conclusion, the design and experimentation plans are scientifically solid. The team is not just throwing in two fancy techniques blindly; they have clear rationales and a plan to evaluate each addition's contribution. The validity of their approach will ultimately be proven by the experimental results, but all signs indicate they are setting up those experiments in the right way to ensure credible, interpretable outcomes. There's a good balance of innovation and use of known methods, which is ideal for a paper: it stands on the shoulders of giants (uses recent techniques) but applies them in a new way to a new problem, likely yielding new insights. As long as they execute the training carefully and avoid pitfalls like overfitting or bug-induced improvements, the project should produce scientifically valuable results.

Recommendations and Improvements

Drawing on experience with transformer architectures and MoE systems, here are several recommendations to further improve the model, training procedure, and evaluation, as well as suggestions to enhance robustness and align the work with top-tier publication standards:

Architectural Enhancements

- **Consider Relative Temporal Biases:** The model now uses RoPE for relative positioning, which is great. Another idea specific to healthcare is to incorporate actual *time gaps* between events as part of the model. For example, if days are not equally spaced (perhaps some patients skip visits for months), the model could benefit from knowing "how much time passed between Day i and Day j ". RoPE handles relative position in terms of index in sequence, but if the data has irregular intervals, you might augment the input with a learned embedding or scalar for "delta days since last visit". This can help the model understand long gaps vs short gaps explicitly. Some prior works in EHR (like chronology modeling) have done this by, e.g., adding a "time difference" embedding or using a modified attention that includes a time decay. It might or might not be needed depending on your data regularity, but it's something to consider if you suspect the model needs to differentiate, say, 1-day apart events from 30-days apart beyond what RoPE provides.
- **Enhanced Code Embedding Fusion:** In the daily encoder, you currently sum the code embeddings as one part of the day representation (a residual) and also use a Transformer + max-pooling. One possible extension is to allow the model to *learn* how to combine those sources instead of a fixed sum. For instance, you could have a small gated mechanism: $h_{\text{day}} = W_1(\text{pooled}_i) + W_2(\text{sum}_i)$ for each day, or even a learned scalar weight between the two (like $\alpha * \text{pooled} + (1-\alpha) * \text{sum}$, with α learned). This could let the model decide how much to rely on the contextualized representation vs. the raw sum. It's a minor tweak and the sum is already giving a strong signal, so it may not change much – but in case the Transformer at the code level is doing a good job, the model might learn to trust it more, whereas if it's not, it would lean on the sum. Essentially, make that residual connection learnable. Another approach is to incorporate the **max-pooled output and sum as separate inputs to the next layer**, rather than summing them

outright, and let the next LayerNorm or projection combine them. However, given this adds complexity and the current method is working, this is an optional refinement.

- **Multi-Head vs Single-Head Daily Encoder:** The daily Transformer uses 4 heads and no positional encoding (since order of codes might be irrelevant or implicitly given by code indices). If the order of codes within a day doesn't matter (likely it doesn't, since often they're a set), you might even consider a simpler *set encoder* for codes – for example, a parameter-free approach like just using mean or max (you already do that partially) or using a **self-attention with learned pairwise relationships** but no positional info. The current approach is fine, but it's worth verifying if that one Transformer layer is actually helping beyond the sum. An experiment could be: remove the code-level Transformer (just sum or average the embeddings per day, then feed to temporal transformer) and see if performance drops. If it doesn't drop much, it might mean that intra-day interactions weren't that crucial or could be captured by a simpler mechanism. If it does drop, then consider if one layer is enough or if perhaps 2 layers at the code level might capture more within-day structure (though that increases cost). Since your main focus is on temporal modeling, I wouldn't complicate the intra-day part unless necessary. But it's something to keep in mind if you find the model struggling to understand combinations of codes on the same day (for instance, certain diagnoses together might imply something that individually they don't).
- **Normalization and Initialization:** You've moved to pre-norm, which is good. One observation from Transformer literature is that sometimes a final LayerNorm on the output (before the decoder) can improve stability (especially if pre-norm is used throughout, adding a final norm ensures the output distribution doesn't drift too much). In your code, after the last temporal layer, you do a LayerNorm and dropout ⁸³. That likely serves this purpose. I'd keep that in place. Another trick: **scaling the initialization** of the MoE experts. You might have done something similar to the FlashTransformer init (like setting output projection weights to zero initially) for experts. If not, it might help to initialize expert feed-forward weights a bit smaller than default, to avoid any one expert producing huge outputs at the start. The reasoning is that if at initialization one expert by chance outputs values much larger than others, the router might latch onto it. Keeping them all near zero output initially makes the router's initial decisions more arbitrary and gives chance for all experts to be tried. In practice, something like fan-in init or Xavier with a small gain could suffice. Since you set router weights small normal (std 0.01) ⁸⁴, you already encourage initially near-equal routing probabilities. That plus small expert weights (maybe already by PyTorch default or you can enforce) would make training start smooth.
- **Alternate Attention Mechanisms:** FlashAttention is basically an efficiency improvement on full attention. In case you consider even longer sequences or want to experiment, there are also *sparse attention* mechanisms (like Longformer, Linformer, Performer, etc.). Your flash redesign doc even mentions these alternatives ³⁸. For 200 days, you probably don't need sparse attention (full attention is fine). But if you did want to scale to thousands of days (perhaps using more coarse time units like hours, etc.), you might explore local attention (sliding window) or a mixture of local and global attention. The redesign doc had an Option 2: Local Attention for ultra-long sequences ⁸⁵. Since you have FlashAttention, you likely won't need these in current scope, but it's good to be aware. If the model ever needs to handle a patient's entire record with thousands of entries, a hierarchical model plus maybe a Longformer-like attention pattern could be combined (for example, attend more finely to recent days and sparsely to distant past). This is more of a future direction than a current need.

- **Expert Placement:** You chose layers 2-5 for MoE. One thought: monitor if maybe not all those layers equally benefit. It could be that maybe the very middle layers (3-4) do most of the heavy lifting with experts, while layer 2 or 5 might not use full capacity. If you find that some layer's experts are underutilized or not contributing (you could check by ablation – e.g., replace one MoE layer back with dense after training to see if performance changes, or just look at the gating patterns per layer if you log those), you could refine which layers use MoE. For example, it might turn out that only 2 of the 4 layers needed to be MoE to get most of the gain, and others could be dense (to save parameters or simplify). This is an advanced optimization; not necessary if results are good, but a way to analyze the model's use of capacity. Google's research sometimes found that earlier MoE layers did less because the model hadn't developed enough diversity of features yet – but in your case earlier layers are dense anyway. So it might be fine as is.
- **Model Size vs Data Size:** Ensure that the MoE capacity is commensurate with data. If your dataset is not extremely large (say only a few hundred thousand patients or sequences), a huge capacity might overfit. MoE tends to help mostly when data is plentiful or very heterogeneous. If you notice overfitting signs (like training loss much lower than validation, and complexity seems a culprit), you might dial back, maybe use fewer experts. However, MoE often generalizes well if balanced, because each expert essentially sees fewer examples (effectively partitioning the dataset), which can reduce overfitting in a counter-intuitive way (each expert is smaller and focuses on a subset of patterns). So you might actually be fine even with more parameters because the effective capacity per token isn't all active at once. In summary, watch the validation metrics relative to train; if a big gap opens, you might need to regularize or reduce capacity.
- **Task Formulation (Multi-label vs Sequence):** From the metrics, it looks like you are predicting multiple possible codes for each day (like a next-visit prediction scenario). If that's the case, ensure the architecture is best aligned with the task. Currently, you output a distribution over `target_cd_cnt` codes for each day ⁶. If multiple codes can be correct, presumably you treat it as a multi-label problem (potentially with multiple ones in the target). Your use of log-softmax and NLLLoss suggests possibly they choose one “primary” code as the target for that day. If that's an artificial construct, one improvement is to treat it as a set prediction per day. You could use a sigmoid output with binary cross-entropy for each code presence. But that makes evaluation tricky (there's no single “true” for Top-K accuracy – you'd have to measure recall@K among the true set). It appears you already measure “Top-5 accuracy” meaning “did the model's top-5 include at least one of the true codes” ⁷⁸. If multiple codes are true, usually one would measure something like Precision@K or Recall@K. However, the way metrics are phrased (“P(true in top-5)”) suggests exactly that – for each actual code in the true set, check if it's in top-5, and maybe average over codes or visits. Clarifying this and possibly adjusting the loss to directly optimize a multi-label objective could improve training. Sometimes a softmax with one true label is used as a proxy (maybe the most critical code), but then the model isn't directly incentivized to predict the other labels. In short, if multi-label prediction is needed, consider a multi-label training loss. If focusing on one main code per day, ensure that is well-defined (maybe the principal diagnosis or so). This is more of a data/label issue than architecture, but it impacts how you design output and loss layers.
- **Interpretability Modules:** If interpretability is a goal, one could add mechanisms to make the model more explainable. For example, one could output attention scores or route probabilities in a user-friendly way for clinical use (like “this prediction was mostly influenced by expert 7 and events from last month”). There are also research works on attention-based explanations in EHR. While not

necessary for model performance, building a simple attention visualization tool or expert usage tracker for case studies could enhance the impact of the work, especially for a conference like *ACL BioNLP* or a clinical informatics venue, where showing how the model arrives at predictions can be important. In terms of architecture, one might constrain experts to correspond to known groupings (like have an expert for each category of disease codes), but that might hurt performance if the grouping is not optimal. It's usually better to let them learn freely and interpret post hoc.

Training and Optimization Strategies

- **Optimizer Selection:** It appears you might be using SGD with momentum and a cosine schedule (since I saw an `optim.SGD` in code) ⁸⁶. Transformers, especially with normalization, often train faster with Adam or AdamW (Adam with decoupled weight decay). AdamW is the standard for most transformer papers. Unless you have a reason to use SGD (perhaps a very large batch regime or to avoid some Adam issues), I'd recommend using AdamW for the main training, with the weight decay you already have (0.01). AdamW will adapt the learning rates per parameter, which might help for MoE because the router and experts might need different treatment. If you do use AdamW, consider using a slightly lower learning rate for the MoE router parameters specifically (some literature does that, e.g., having a smaller LR on gating to avoid oscillations). But you can also keep it uniform and see – the auxiliary loss and bias correction will guide the router.
- **Learning Rate Schedule and Warmup:** You have a cosine annealing schedule which is good for long training. However, ensure you use a **warmup** period at the start (a few hundred steps of gradually increasing LR from 0 to 1e-4). This is crucial for transformers to avoid early instability. Warmup is especially important when you have random initialization and layer norms – without it, the first few batches might produce large gradients. Warmup combined with gradient clipping will make the start of training smoother. Many transformer training routines use e.g., 10,000 steps warmup for very large data, or a fraction of an epoch. Even a short warmup (say, 5% of an epoch) could help. I noticed `warmup_steps=0` in your config dataclass ⁸⁷ – I strongly suggest making that non-zero.
- **Gradient Clipping:** You set clip norm to 1.0 which is sensible. Monitor if it activates often (if your training logs can show how often gradients are being clipped). If it's hitting frequently, it means perhaps the learning rate is a bit high or something is causing spikes (maybe router changes). That might be okay as long as loss is decreasing. Just keep an eye that clipping doesn't mask an underlying issue (like one expert blowing up – though clipping would address symptoms, not cause). But generally, 1.0 is fine and commonly used.
- **Mixed Precision:** Use of `torch.bfloat16` or `float16` will speed up training. Be vigilant for any overflow or NaN issues. If any arise, you might have to use gradient scaling (with `float16`) or ensure certain parts run in FP32 (like the softmax). PyTorch's `GradScaler` usually handles FP16. Bfloat16 is more robust (no grad scaling needed typically). If you have Nvidia A100s or higher that support BF16, that's ideal. T4/V100 might not support BF16 as well, so you'd use FP16 there. I suspect the environment has Ampere or better (given flash attn), so BF16 is likely available. It's good that you considered this in the design. It shouldn't change model quality, it just speeds it up and reduces memory.

- **Regularization and Dropout:** You use dropout 0.1 in many places, which is standard. With MoE, sometimes a bit more dropout can be useful because the model has many parameters (e.g., Switch Transformer used Dropout up to 0.1-0.2). If you notice overfitting, increasing dropout to 0.2 in FFNs or attention might help. You also have an `expert_dropout=0.05` in config for experts ⁸⁸ – presumably that means you apply dropout inside each expert’s FFN. That’s good for regularizing each expert. There’s also the concept of **router dropout** or **expert dropout** where during training you randomly drop out some expert routes to force redundancy. For example, during a batch, you could randomly ignore the second expert occasionally (making it effectively top-1 for some tokens) or drop one expert entirely for some batch. This was suggested in some MoE works to make sure no single expert is absolutely required. It’s a bit complicated to implement and might not be necessary. But if you see any expert is dominating or the model becomes too reliant on a combination of experts, such noise injection could help. Only consider this if you run into issues like expert collapse that aux loss didn’t solve.

- **MoE Hyperparameters:** You might need to adjust:

- *Aux loss weight (0.01):* If you find that experts are still imbalanced, you might raise it slightly. If you find it’s hurting main loss (like model converges slower or to worse accuracy), you could lower it or rely on bias. Often 0.01 is okay, but occasionally tasks use 0.001 or 0.1 depending on how severe imbalance is vs how much it interferes.
- *Bias LR (1e-5):* The DeepSeek bias update learning rate is critical – too high and it will oscillate, too low and it won’t correct in time. 1e-5 with momentum 0.9 sounds reasonable (it will slowly adjust biases). You might have to tune this by looking at how the loads evolve. If after some epochs you still see imbalance, maybe increase it. If you see it overshooting (like an expert’s bias becomes very negative and then usage swings), maybe decrease it or momentum.
- *Top-k (try k=1?):* Not in the main experiments, but as an ablation it might be interesting to see if top-1 expert (like a Switch Transformer variant) performs significantly worse or not. If it’s close, one could argue top-1 might suffice, which would halve the expert compute. But I suspect top-2 will be better given the data complexity. Not a priority, but something to confirm if you have time.
- *Shared expert usage:* If using a shared expert, ensure its capacity is not too large such that it trivializes the MoE. You did keep it same `d_model` and `d_ff`, which is fine. Just check after training whether the shared expert carries a lot of load (since it’s always used, how much does it contribute?). One way is to compare the magnitude of its output vs the routed ones. If the shared expert output is consistently, say, an order of magnitude larger than the others, it might be dominating the representation (which isn’t necessarily bad if that’s optimal). But if it dominates, then effectively the model became more like a dense model plus some minor adjustments from others. If that happens, you might need to regularize it (maybe reduce its size or add an aux loss to not put all variance in shared). This is speculative – the shared might just learn broad strokes and let others fine-tune per token.
- **Evaluation During Training:** Monitor your primary metrics (like validation top-5 accuracy or NLL) during training. With MoE, sometimes you see training NLL drop faster than baseline (because more capacity), but validation might not improve as much initially – maybe needing more epoch to realize benefit or risk of overfitting. If you see overfitting, you could employ early stopping on validation loss or accuracy. The plan is to evaluate each epoch (`eval_frequency=1`) ⁸⁹ which is good. If training

is long, maybe evaluate every few epochs to save time, but given you want to catch when to stop, each epoch or two is fine.

- **Epochs and Convergence:** You put `epochs=1` in config, but likely you'll train for more (perhaps that was a placeholder for initial tests). Ensure you train sufficiently for each experiment. It might take several epochs for MoE to fully utilize experts properly (the aux loss will make initial progress slow until balance is achieved). If baseline needed, say, 10 epochs to converge, MoE might need similar or a bit more. Watch if the router probabilities continue shifting late into training; if so, maybe more epochs help. Conversely, if it converges quicker due to more capacity, you might see it plateau earlier. Either way, plan to run until validation metrics stop improving plus maybe a little beyond.
- **Data Parallel vs Model Parallel:** You are using DataParallel (replicating model on 4 GPUs and splitting batch). That's straightforward. Just be mindful that in MoE, if one GPU's batch has a weird distribution of tokens, the experts on that GPU still get loaded evenly because each GPU has a full copy of the model. DataParallel doesn't partition experts, so it's fine. If you ever considered using Model Parallel for MoE (assign different experts to different GPUs to increase number of experts beyond memory of one GPU), that's a more complex approach – not needed here since 8 experts fit on one GPU easily. DataParallel will result in each GPU computing all experts for its batch segment. This is okay as long as your batch per GPU (4 in case of total 16) is not so small as to cause imbalance noise. With batch 4 per GPU, the router decisions might be a bit unstable (because with only 4 sequences, maybe not every expert gets tokens each update). If you can increase batch size per GPU, that might stabilize MoE training. Perhaps you could run batch 32 on 4 GPUs (8 per GPU) if memory allows. If not, it's not a deal-breaker, but small batch MoE training can be noisier in balancing. One trick if batch can't be large: accumulate gradients over multiple steps effectively to simulate a larger batch for the router. But that complicates training somewhat.
- **Loss Function Tweaks:** If you stick to NLL (with one target code per sequence per day), you might consider **label smoothing** as a regularizer. For example, giving a small probability (say 0.1) mass to non-targets. This can help the model not become overconfident and usually improves generalization slightly, especially in classification tasks with many classes (you have thousands of possible codes). It's used in many Transformer models (like in the original Transformer paper they used label smoothing 0.1). It's easy to implement and might improve calibration and maybe a tiny boost in accuracy. However, if you have multiple true codes, label smoothing might not make sense unless you pick one code as true. In multi-label, label smoothing is less applicable.
- **Frozen vs Finetune:** Unlikely relevant here, but if you had any pretrained embeddings or components (say you pre-trained the model on a larger dataset), then appropriate freezing or low LR for those would be needed. It sounds like you are training from scratch on your data, so not applicable. But if, for example, you had code embeddings from another model (like word2vec for medical codes), you could initialize the embedding layer with those. Given the large vocab (84k codes), pretraining embeddings might help if data is limited. If data is plenty (millions of records), it's probably fine training from scratch with proper initialization.

Experimental Validation Enhancements

- **Baseline Comparisons:** Make sure to compare with strong baselines. Apart from your own baseline hierarchical model, consider a non-hierarchical Transformer baseline if feasible (even if you have to truncate sequences). Also consider classical methods or simpler deep models:
- For instance, an LSTM-based model or GRU that goes through days (with codes aggregated per day). Or RETAIN (a popular RNN with attention for EHR). If they perform far worse, it strengthens your case that the Transformer architecture (with or without MoE) is superior.
- If any published numbers exist for similar tasks, include them. E.g., if BEHRT or another model achieved X top-10 accuracy on some dataset comparable to yours, mention that and ideally evaluate your model on the same if possible.
- **A parameter-matched dense Transformer:** e.g., if your MoE model has ~30M parameters, you could try a dense model with 30M (maybe by increasing FFN dim or number of layers) to see if MoE is more parameter-efficient. This addresses the question: could we just make the model bigger (without MoE) and get the same benefit? If MoE outperforms a dense model of similar size, that's a key result (sparsity yields better utilization of parameters). If a dense model matches it, the benefit is more about capacity than specialization, but even then MoE provided a way to get that capacity at lower runtime cost. So either outcome can be framed positively, but obviously MoE truly shining would be best.
- **Ablate Flash improvements:** It might be informative to run the baseline with and without the flash-related enhancements (RoPE, pre-norm, etc.) to quantify their effect. For example, baseline original vs baseline+RoPE, or baseline vs baseline+16 heads (in case original had 8). This helps disentangle whether improvements come partly from those architectural tweaks. Since you likely will incorporate all improvements into the final model, doing a full ablation grid might be too much. However, at least mention or test that e.g., adding RoPE and pre-norm to the baseline didn't degrade performance (they should improve or equal). If they improve baseline a bit, then your MoE improvements are on an already stronger baseline, which is good.
- **Statistical Significance and Variance:** For each experiment, try to run a couple of random seeds if time permits. MoE training can have variance due to the non-deterministic routing (especially if multi-threaded ops or any randomness in tie-breaking for top-k). Ideally, report mean and std for key metrics. If that's not possible, be cautious in claiming improvements under, say, 1-2% because they might not be significant. But if improvements are large (>5%), one run might suffice to demonstrate it. Still, for a paper, showing consistency is very valuable. You could perhaps train the baseline and one of the MoE configs 3 times and show error bars.
- **Complexity & Efficiency Metrics:** Since one of your selling points is efficiency (FlashAttention), include some empirical measures:
 - Training speed: e.g., "with FlashAttention, training is **X% faster** and uses **Y% less GPU memory** than the baseline implementation" – you likely have data from your benchmarking script ⁹⁰. For instance, measure the time per batch or memory usage per model. This quantifies the benefit of the flash integration.
 - Inference speed: If this is aimed for deployment, show that at inference the model can process so many sequences per second on a given hardware. Flash attention will help here too. If MoE had any overhead, mention it. However, since MoE doesn't increase sequence length, the main overhead is the second expert computation which is linear. So likely the MoE model might be, say, 1.5× slower

than dense in inference. But if that's still within acceptable range, it's fine. If needed, you can also run a batch through baseline vs MoE and measure latency.

- Perhaps include a *FLOPs* or operations count analysis. E.g., "Baseline model has ~D million parameters and executes ~F gigaflops per sequence. Our MoE model has ~D2 million parameters but, thanks to sparsity, executes ~F2 gigaflops per sequence (which is only moderately higher than baseline, or maybe even equal if we compare to a larger dense model of similar param count)." This kind of analysis can bolster the argument that you achieved a favorable accuracy-compute trade-off.

- **Robustness Testing:** After training, test the model on some edge cases:

- Patients with very short history (e.g., only 1 day of data). Does the model handle that gracefully? (It should, due to padding and mask).
- Patients with very long history beyond training length if possible (to test RoPE extrapolation). For example, if you can get a patient with 220 days and see if the model still performs sensibly (maybe using a model trained on 200-day max). This might be outside your data distribution, so it's more a curiosity – but demonstrating extrapolation would be neat (even if just qualitatively).
- Different subgroups: You could break validation results by age group or by disease category to see if the model does uniformly well. If you find, for example, that it struggles on pediatric patients or something, that might be a finding to discuss. Or MoE might have fixed some subgroup performance issues. This can be presented in a paper's appendix or narrative if relevant.
- Out-of-distribution visits: If there's a way to simulate or get a test set from another source (e.g., another hospital or time period), it can show generalization. That might be beyond scope if data is limited to one source. But even temporal split (train on older data, test on newer data) could be an interesting robustness check. Models that rely on specific frequencies might degrade; MoE might help if rare new patterns appear (since an expert could adapt).

- **Interpretability and Analysis:**

- Analyze **expert specialization** after training. One approach: for each expert, find which codes or situations strongly activate it. Since you have the router probabilities, you could, for a given expert, gather all tokens (day representations) where that expert was chosen in top-1 or top-2. Then analyze what those tokens have in common. Perhaps expert 3 is frequently used for patients with a certain diagnosis history. You could even train a simple decision tree on the input features to predict when expert 3 is chosen, to approximate its "responsibility". Alternatively, look at the output of that expert's FFN: maybe it has certain weight patterns that correspond to particular codes. This might be tough to parse directly, but maybe you can compute for each expert's output layer, which target codes it most strongly affects (since each expert output goes through the shared output layer eventually, but if you had separate output heads per expert in some analysis scenario you could see that).
- Visualize an example: pick a patient sequence and illustrate which days attended to which previous days (from the attention matrices) and which experts were used for each day. This could be a powerful figure for a paper or presentation: e.g., "In this heart failure patient's timeline, we see that early in the sequence (when the patient's conditions were mild), general expert and expert A were used, but later when complications arose, the model increasingly routed to expert B which specializes in complex multi-morbidity patterns." And attention might show that the prediction at a

certain day looked back strongly to an event 5 days prior, etc. Even if anecdotal, it provides intuition that the model is doing sensible things.

- **Calibration:** Especially in healthcare, you want the probability outputs to be somewhat calibrated (so that a predicted probability of 0.8 corresponds to ~80% chance in reality). Transformers can be overconfident. With MoE, this could be even trickier (since each expert might be confident in its domain). Consider evaluating calibration (e.g., reliability diagrams) and possibly apply temperature scaling to the final outputs if needed. This is not mandatory for a modeling paper, but if you aim to deploy, it's important for clinical use (so that the care providers can trust the probability estimates). Some metrics like Brier score or ECE (Expected Calibration Error) could be computed for your predictions as well.
- **Downstream Task Evaluation:** You mentioned external evaluation, which likely means taking the learned representations and using them for some supervised tasks. Definitely do this if you can:
 - For example, use the final hidden state of the sequence (or perhaps a pooled representation of all days, maybe via another attention) as a patient embedding, and train a simple classifier on a task like predicting a diagnosis or outcome at a certain time horizon. See if using the MoE-enhanced model's embeddings yields better performance than baseline's embeddings. This would show that the model not only predicts next codes well, but learns features useful for other predictions (which is a claim of general usefulness).
 - Another external task could be clustering patients based on their embedding to see if clinically meaningful clusters emerge (e.g., do patients cluster by disease subtype). That's more of an exploratory analysis but can be a nice insight to mention.
 - If your model is trained in a self-supervised way (predicting next codes), you can fine-tune it for a specific supervised task and see if it converges faster or to a higher score than training that task from scratch. That would position your model as a *pretrained foundation model for EHR* – which is a very hot topic.

Robustness, Generalization, and Interpretability

- **Robustness to Rare Events:** One of the motivations for MoE is to handle rare codes better ⁶². Make sure to verify this. You have a metric for rare code Top-10 accuracy ⁸¹ ⁹¹. If MoE significantly improves this, highlight it. If not, analyze why – perhaps the model still didn't get enough signal for some codes. You might then consider techniques like upsampling rare events in training or using focal loss (which focuses on hard examples) to further boost performance on rare codes. But those would complicate training; hopefully MoE does the job by itself. Nonetheless, verifying and possibly addressing any residual issues with rare code prediction will strengthen claims of robustness.
- **Generalization to Longer Sequences:** As noted, test extrapolation if possible. If the model can handle sequences longer than seen, that's a nice robustness feature. RoPE theoretically allows it, but sometimes performance drops if asked to extrapolate too far because model weights weren't optimized for that regime. If you can't test longer sequences on real data, maybe a synthetic test: feed the model a copy of a sequence appended to itself (so 400 length) and see if it still produces reasonable outputs or at least doesn't error out.
- **Out-of-vocabulary and Missing Data:** If a patient has a code that was never seen in training (possible if you split data or if new codes appear), how does the model handle it? The embedding layer size is fixed to training vocab (84010 codes). If a truly new code appears, you currently have no embedding for it. Typically, one would have an "unknown" token embedding for codes not in vocab.

If you haven't, consider adding a catch-all OOV embedding for any code ID that's out of range. Or if your data's closed vocabulary, not an issue. Also, ensure that if a patient day has no codes (maybe a visit record exists but no code? or days between visits), how is that represented? Possibly as a zero-padding which your model already handles (empty day would be a vector of zeros after sum and transformer). That should be fine. Just verify that a completely empty sequence (no visits) is handled (probably not needed since you likely filter those out).

- **Fairness:** Since the model uses demographics as input, it's explicitly accounting for age and gender which is good for accuracy, but one must be mindful in some contexts that it could also reinforce biases (for example, if certain diagnoses are under-recorded in one gender, the model might rely on gender too heavily in prediction). Fairness in clinical predictions is a big topic. A suggestion is to evaluate performance across subgroups. You can check if, say, Top-5 accuracy for females vs males are similar. If there's a gap, investigate why. It might be fine; including gender often actually improves performance for both genders if used properly. But if an issue arises (like model systematically underpredicts a condition in one group), you might need to adjust (perhaps by balancing training data or adding group-specific experts – interestingly, the MoE might naturally create an expert that handles one group if needed, which would be a fascinating finding). For age, similarly check if the model's performance drops for very young or very old patients differently. Ensuring robust performance across demographics will make the model more trustworthy for deployment.
- **Model Uncertainty:** In a critical domain, knowing when the model is unsure is valuable. You could implement a simple uncertainty estimate by looking at the entropy of the output distribution or the gap between the top prediction probabilities. If the model gives a very flat distribution, it's not confident. If MoE is functioning, one could also see if experts strongly disagree (e.g., if top-1 and top-2 experts produce very different outputs and the router is 50/50 between them, that could indicate uncertainty or an input that lies between learned patterns). This could be leveraged to say "for these cases, the model has lower confidence, maybe a human should double-check." It's a bit advanced to integrate into the pipeline, but even analyzing a few cases with low confidence might reveal if they correspond to unusual patients.
- **Production readiness:**
 - Packaging the model: after training, you may want to combine the separate code into one model class or pipeline. For example, right now you might have the code embedding and daily encoder and temporal layers somewhat separate (depending on how you refactored). Ensure that it's straightforward to serialize the model and load it for inference. This typically means avoiding any custom Python logic that can't be easily saved. Using standard `nn.Module` as you did is good. Test saving and loading the `FlashTransformerModel` with `torch.save` and `torch.load` to ensure no issues.
 - **Resource usage:** If deploying in a hospital setting, one might not have a GPU on hand for real-time inference. If CPU inference is needed, see how slow it is. Transformers on CPU are slower but maybe okay for moderate sequence lengths if multi-threaded. MoE on CPU could be slow with Python loops, but one could compile it with TorchScript to C++ which might help. If the intended use is offline batch prediction, then speed isn't as critical; if it's online (like providing suggestions to doctors as they input data), latency should be within a few seconds at most. Possibly beyond the scope of research to fully address, but be aware of where the model would run.
 - Ensuring **compatibility** with clinical IT: That's more a deployment detail – e.g., one might have to integrate the model with an EHR system. That means reading data in the right format and outputting predictions in a format that the system can use (like a list of suggested codes). In terms

of your code, making sure there's a clear interface (input = patient records, output = list of predicted codes with probabilities) will facilitate integration.

- **Monitoring** in production: If you deploy, set up logs for expert usage, etc. as mentioned, to detect drift or anomalies. For example, if suddenly expert 8 usage goes from 10% to 50% on new data, something changed – maybe a new kind of patient or a data error. That can alert you to retrain or adjust.
- **Continual Learning:** The field of LLMs and MoE suggests that MoE is great for continual learning (you can add experts to learn new data without interfering with old ones too much). While your project might not cover this, you could mention it as a future advantage: e.g., “In the future, as new medical codes or practices emerge, we could add or fine-tune specific experts rather than retraining the whole model, potentially making updates more efficient.” This is a selling point especially to a conference like NeurIPS that appreciates forward-looking statements about scalability and maintainability of models.

Aligning with Top-Conference Standards

- **Emphasize Contributions:** In your write-up and presentation, clearly state *what is novel*. For example:
 - “We introduce a **hierarchical MoE Transformer** for clinical sequences, which is the first to apply mixture-of-experts to EHR modeling (to our knowledge).” Even if not the absolute first, if it's novel enough, claim it carefully.
 - “We integrate FlashAttention and demonstrate its benefits in a real-world data setting (reducing training time and enabling longer sequence modeling than prior EHR Transformers).”
 - “We conduct an extensive ablation study isolating the effects of experts count, shared experts, and balancing mechanisms, which provides insights into efficient model scaling for this domain.”
 - “We show that our model improves prediction of clinically important but rare events by X% over strong baselines, and produces more calibrated predictions for common events.”

Having 3-4 bullet points like these in the introduction of a paper will help reviewers quickly grasp why your paper is valuable.

- **Related Work:** Ensure you cite relevant prior work:
 - Transformers in healthcare: BEHRT (Li et al, Scientific Reports 2020) ⁹², Med-BERT, GatorTron (if any large clinical transformers), ClinicalBERT (though BERT is different type of usage), RETAIN (Choi et al 2016, an RNN with attention) as a non-transformer baseline, Hi-BEHRT (Zhu et al 2020) which you cited in research, etc. This situates your work as building on the progression of models for EHR.
 - Mixture-of-Experts: Cite Switch Transformer (Fedus 2021), the original MoE paper (Shazeer 2017 if relevant), maybe recent works like BASE Layer (Lewis 2021) or Vishvanathan et al's ST-MoE (Zoph 2022) ⁹³ which might be that reference to “ST-MoE (Zoph 2022)” in your metrics table. This shows you know the MoE landscape. Also cite the DeepSeek tech report if you can (though if it's not on arXiv yet, cite as unpublished or arXiv preprint 2024).
 - FlashAttention: Cite the 2022 paper and perhaps FlashAttention-2 (Dao et al 2023) if relevant (though FlashAttn 2 is more about long sequences > 1k).
 - If any efficient transformer was used in EHR context (maybe not yet, but if there is any, mention them).

By discussing how these works influenced your design (e.g., “Unlike BEHRT which uses standard attention and struggles beyond 512 tokens ⁹⁴, we leverage FlashAttention to handle longer sequences efficiently. And whereas Switch Transformer was demonstrated on NLP tasks, we show MoE’s benefits on clinical time series, with domain-specific insights like improved rare event prediction.”), you make it clear that you’re advancing the state of the art.

- **Writing Clarity:** From the quality of your internal docs, it seems you’re adept at writing clearly. For conferences, make sure to include necessary figures (like model architecture diagram) and tables (results, hyperparameters in appendix etc.). The *Table 1 and 2* you drafted in the plan are excellent and can go straight into a paper to describe experiments ⁶⁶ ⁹⁵. Just ensure to condense where needed for page limits (NeurIPS/ICLR allow ~9-10 pages main content). If space is tight, some detailed ablations might go to appendix, but try to keep key results in main paper.
- **Reproducibility:** Provide enough detail on data (cohort size, how codes are processed, etc.) so others can attempt to replicate. If data is proprietary, at least describe it well. Perhaps release code on GitHub (with any sensitive info removed) to show transparency – many conferences like to see a statement that code will be released. Since you already have a nicely modular codebase, that should be doable. You might even consider releasing a synthetic dataset or a trained model (if allowed by data privacy rules) for the community.
- **Ethical/Impact Statement:** NeurIPS and others require a discussion of societal impact. For a clinical prediction model, note potential positive impacts (better patient outcomes through earlier detection, assisting doctors, etc.) and risks (automation bias, privacy concerns, potential bias as discussed). Clarify that this model is a decision support tool and not a final arbiter of care, to mitigate concerns. Also mention that patient data was handled securely and any biases in data could affect model decisions, which you have to monitor.
- **Presentation:** If you target an ML conference, focus on the ML innovations and insights. For instance, highlight how the expert gating found structure in the data (maybe refer to an interesting finding like “Expert 5 primarily activates for oncology patients, showing the model discovered a subpopulation-specific pattern”). Those kind of insights are gold in a paper because they show the model not only improved metrics but taught us something about the data or modeling technique. If you have any negative results (e.g., “We tried top-4 experts and saw no further improvement, likely due to diminishing returns in capacity usage”), mention them briefly to save others the trouble – this shows thoroughness.
- **Topical Fit:** Your work sits at the intersection of *efficient model architectures* and *healthcare AI*. NeurIPS/ICLR will appreciate the former; venues like *Machine Learning for Health (ML4H)* workshop or *ACM CHIL*, etc., will appreciate the latter more. If going for a general ML conference, stress the generalizable aspects: e.g., “Our hierarchical MoE approach could be applied to any scenario with hierarchical sequence data (e.g., sessions of user activities composed of events) and long-range dependencies, not just healthcare.” That broadens the impact. If going for a more healthcare-focused venue, emphasize the domain-specific gains (handling long patient histories, rare conditions, etc.). It sounds like you’re aiming high (NeurIPS/ICLR), so make sure the paper appeals to the ML crowd: focus on modeling and algorithmic contributions, and use the healthcare scenario as the proving ground.

In summary, the project is on a strong track. By implementing these recommendations, you can refine both the model's performance and the way you convey its value: - Architecturally, just a few minor tweaks (learnable combo of embeddings, maybe time-gap encoding) could be considered, but the main components are wisely chosen. - Training-wise, using the best optimization practices (AdamW, warmup, etc.) and careful regularization will ensure the model reaches its full potential without instability. - Experimentally, adding comparisons, significance analysis, and diving deep into *why* the model works (expert analysis, etc.) will elevate the work from just "we built a model that's better" to "we understand what makes a model better for this problem". - For robustness and deployment, checking fairness and reliability will be important if this is to be used in a clinical setting, and it will also make the work stand out as thorough and responsible. - When writing the paper, highlight both the methodological advances (e.g., first use of MoE in this context, the effective integration of flash attention) and the empirical results (state-of-the-art performance on tasks, new insights into modeling EHR sequences).

By addressing these points, you will not only improve the model and its evaluation but also demonstrate to reviewers that you have done a **comprehensive, rigorous study**. This increases the chances of acceptance at a top conference and sets a high standard for others working on similar problems. Good luck with training and we look forward to seeing the impactful results of this work!

Sources:

- Hierarchical Transformer model for EHR (Hi-BEHRT) and the motivation for hierarchical structure 38
- Implementation details of Flash Attention and modern Transformer layers 9 12
- Two-level architecture and insertion point for MoE in temporal layers 4 96
- MoE routing mechanism and load balancing as implemented 23 25
- Experimental plan outlining variables controlled vs tested across Exp 1-5 20 71
- Metrics chosen for evaluation, emphasizing top-K accuracy and rare code performance 78 62
- Metrics to monitor MoE behavior (load, entropy) to ensure balanced usage 59
- FlashAttention memory advantage (no big attention matrix) leading to efficiency 7 8
- Pre-normalization and other optimizations in the custom Transformer layer 10 97
- Router initialization and gating strategy in MoE (small init, softmax top-k) 21 98
- Shared expert output being always added for each token 26 27
- Switch Transformer aux loss formula and goal of equalizing expert load 28 29
- DeepSeek bias approach for load balancing (adjusting router logits) 32
- Expected improvements from each experiment as per the plan (+5-8% for MoE, etc.) 61 68
- Controlled experiment design enabling attribution of improvements 67 69

1 4 19 20 42 43 59 61 62 66 67 68 69 70 71 75 76 77 78 79 80 81 82 91 93 95 96

MOE_experiment_plan_sameFLOPs.md

file:///file_000000008a3c7230b4a573395d7f7690

2 3 5 6 11 14 17 18 51 52 53 83 90 **flash_attention_implementation_plan.md**

file:///file_00000000286472308155fe6fb8bf9733

7 8 9 12 16 44 45 46 48 49 50 55 85 97 **flash_attention_redesign.md**

file:///file_00000000737c723093d8add77793c732

10 13 15 47 54 **flash_attention.py**

file:///file_0000000088247230948290b254f52dfb

21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 56 57 58 60 63 64 65 72 73 74 84 87 88

89 98 **moe_1.py**

file:///file_00000000154072308278cf99be904676

38 92 94 **Hi-BEHRT: Hierarchical Transformer-Based Model for Accurate Prediction of Clinical Events Using Multimodal Longitudinal Electronic Health Records - PMC**

<https://pmc.ncbi.nlm.nih.gov/articles/PMC7615082/>

39 40 41 86 **min_transformer_finetune.py**

file:///file_00000000dd687230a24dead269ec0de0